

FORM-TL-04 — Taller — Diseño de Esquema de Fragmentación y Replicación de BDD

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación
Ingeniería de Software — Aplicaciones Distribuidas

07/07/2026

Estudiante:

Jhinson Stalyn Aucatoma Celorio

1. Introducción

El presente informe muestra el diseño e implementación de un esquema de fragmentación de bases de datos distribuidas, aplicado sobre un escenario de una cafetería universitaria con tres sedes (Campus, Babahoyo y Ventanas). Se emplean tres nodos PostgreSQL independientes, desplegados como contenedores Docker, sobre los cuales se aplican fragmentación horizontal, vertical y mixta, y se reconstruyen los datos mediante vistas globales apoyadas en `postgres_fdw`. Finalmente, se verifican formalmente las tres condiciones de corrección que debe cumplir toda fragmentación: completitud, reconstrucción y disjunción.

2. Preparación del entorno de trabajo

Se utilizaron tres contenedores Docker, cada uno ejecutando una instancia independiente de PostgreSQL 16, representando las tres sedes de la cafetería. La orquestación se definió mediante `docker-compose.yml`.

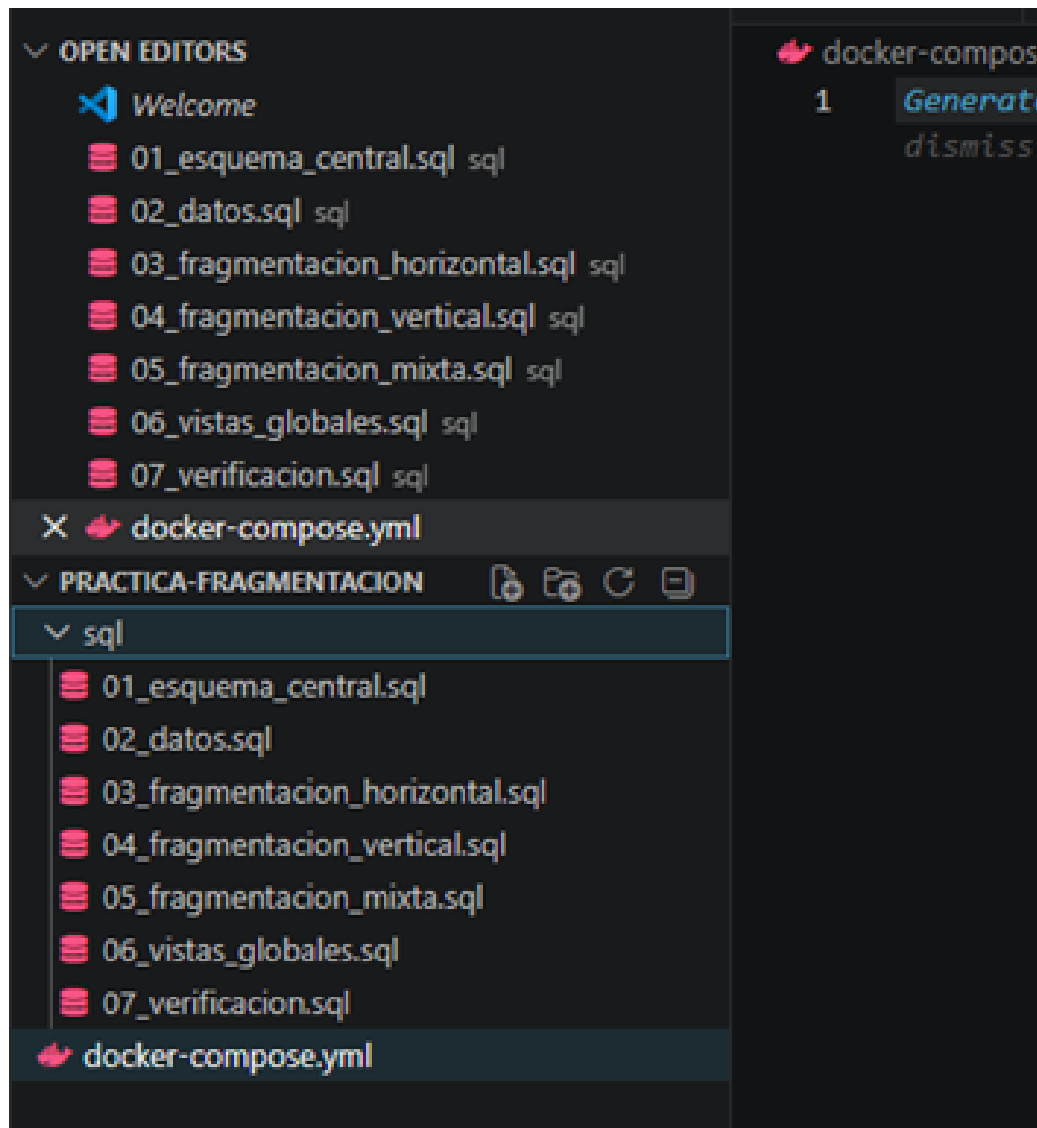


Figura 1: Estructura de carpetas del proyecto.

Listing 1: docker-compose.yml — tres nodos PostgreSQL, uno por sede

```

1 services:
2   pg-campus:
3     image: postgres:16-alpine
4     container_name: pg-campus
5     environment:
6       POSTGRES_DB: cafeteria
7       POSTGRES_USER: admin
8       POSTGRES_PASSWORD: admin123
9     ports: ["5433:5432"]
10    volumes:
11      - pg-campus-data:/var/lib/postgresql/data
12
13   pg-babahoyo:
14     image: postgres:16-alpine
15     container_name: pg-babahoyo
16     environment:
17       POSTGRES_DB: cafeteria
18       POSTGRES_USER: admin

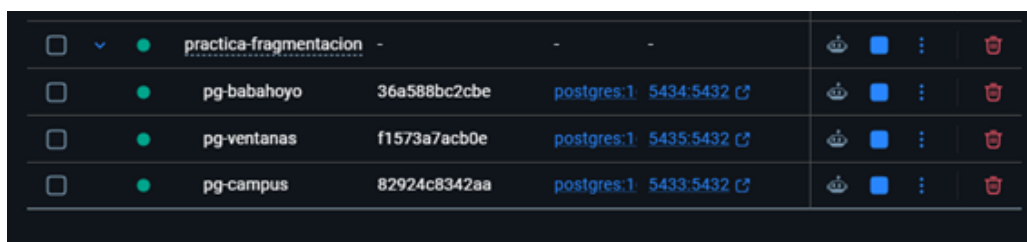
```

```

19     POSTGRES_PASSWORD: admin123
20     ports: ["5434:5432"]
21     volumes:
22     - pg-babahoyo-data:/var/lib/postgresql/data
23
24     pg-ventanas:
25     image: postgres:16-alpine
26     container_name: pg-ventanas
27     environment:
28     POSTGRES_DB: cafeteria
29     POSTGRES_USER: admin
30     POSTGRES_PASSWORD: admin123
31     ports: ["5435:5432"]
32     volumes:
33     - pg-ventanas-data:/var/lib/postgresql/data
34
35 volumes:
36     pg-campus-data:
37     pg-babahoyo-data:
38     pg-ventanas-data:

```

Tras ejecutar `docker compose up -d`, se verificó que los tres contenedores quedaran en estado *running*.



Container Name	ID	Image	Ports	Status
practica-fragmentacion	36a588bc2cbe	postgres:16-alpine	5434:5432	Running
pg-babahoyo	f1573a7acb0e	postgres:16-alpine	5435:5432	Running
pg-ventanas	82924c8342aa	postgres:16-alpine	5433:5432	Running

Figura 2: Los tres nodos PostgreSQL en ejecución, verificados con docker desktop.

3. Esquema centralizado

Antes de fragmentar, se creó el esquema completo en un solo nodo de referencia (`pg-campus`). Este esquema define tres entidades relacionadas: `clientes`, `productos` y `pedidos`. La tabla `pedidos` actúa como entidad intermedia entre las otras dos, con dos claves foráneas (`cliente_id`, `producto_id`) que garantizan integridad referencial. Se incluyó la columna `sede` en `pedidos` porque es precisamente el criterio que se usaría más adelante para la fragmentación horizontal: cada pedido queda asociado a la sede donde se generó.

Listing 2: 01_esquema_central.sql

```

1 CREATE TABLE clientes (
2     cliente_id SERIAL PRIMARY KEY,
3     nombre VARCHAR(80) NOT NULL,
4     ciudad VARCHAR(40) NOT NULL,
5     email VARCHAR(120) NOT NULL,
6     telefono VARCHAR(20)
7 );
8
9 CREATE TABLE productos (
10     producto_id SERIAL PRIMARY KEY,
11     nombre VARCHAR(80) NOT NULL,
12     categoria VARCHAR(30) NOT NULL,

```

```

13     precio NUMERIC(6,2) NOT NULL CHECK (precio >= 0)
14 );
15
16 CREATE TABLE pedidos (
17     pedido_id SERIAL PRIMARY KEY,
18     cliente_id INTEGER NOT NULL REFERENCES clientes(cliente_id),
19     producto_id INTEGER NOT NULL REFERENCES productos(producto_id),
20     fecha DATE NOT NULL,
21     monto NUMERIC(8,2) NOT NULL CHECK (monto >= 0),
22     sede VARCHAR(20) NOT NULL
23 );

```

Este esquema se aplicó únicamente en pg-campus, ya que representa la “fuente de verdad” contra la cual se compararán los fragmentos en la etapa de verificación (sección 9).

```

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-campus psql -U admin -d cafeteria
psql (16.14)
Type "help" for help.

cafeteria=# /dt
cafeteria=# \dt
          List of relations
Schema | Name      | Type  | Owner
-----+-----+-----+-----
public | clientes  | table | admin
public | pedidos  | table | admin
public | productos | table | admin
(3 rows)

cafeteria=# _

```

Figura 3: Verificación de la creación exitosa del esquema centralizado en pg-campus.

Diagrama entidad-relación

La Figura 4 muestra el modelo entidad-relación adaptado a los datos reales del equipo: un cliente puede generar muchos pedidos (relación 1:N), y cada pedido incluye exactamente un producto (relación 1:N desde productos).



Figura 4: Modelo entidad-relación adaptado — Cafetería UTEQ.

4. Carga de datos de prueba

Los datos de prueba se diseñaron para que los 6 clientes se distribuyeran entre las tres ciudades (2 en Quevedo, 2 en Babahoyo, 2 en Ventanas), y para que los 8 pedidos cubrieran las tres sedes de forma desigual (3 en Campus, 3 en Babahoyo, 2 en Ventanas). Esta distribución permite observar con claridad cómo cada fragmento recibe un subconjunto distinto y no vacío de filas, condición necesaria para validar completitud y disjunción.

Listing 3: 02_datos.sql (extracto)

```

1 INSERT INTO clientes (nombre, ciudad, email, telefono) VALUES
2 ('Maria Alvarado', 'Quevedo', 'maria@uteq.edu.ec', '0991111111'),
3 ('Luis Cedenó', 'Babahoyo', 'luis@uteq.edu.ec', '0992222222'),
4 ('Ana Vera', 'Quevedo', 'ana@uteq.edu.ec', '0993333333'),
5 ('Jose Mendoza', 'Ventanas', 'jose@uteq.edu.ec', '0994444444'),
6 ('Carla Zambrano', 'Ventanas', 'carla@uteq.edu.ec', '0995555555'),
7 ('Pedro Suarez', 'Babahoyo', 'pedro@uteq.edu.ec', '0996666666');
8
9 INSERT INTO pedidos (cliente_id, producto_id, fecha, monto, sede) VALUES
10 (1, 1, '2026-03-01', 0.75, 'Campus'),
11 (2, 3, '2026-03-01', 2.50, 'Babahoyo'),
12 (3, 2, '2026-03-02', 1.00, 'Campus'),
13 (4, 4, '2026-03-02', 1.25, 'Ventanas'),
14 (5, 5, '2026-03-03', 1.00, 'Ventanas'),
15 (6, 1, '2026-03-03', 0.75, 'Babahoyo'),
16 (1, 3, '2026-03-04', 2.50, 'Campus'),
17 (2, 2, '2026-03-04', 1.00, 'Babahoyo');

```

```

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-campus psql -U admin -d cafeteria
psql (16.14)
Type "help" for help.

cafeteria=# select * from clientes;
 cliente_id | nombre      | ciudad  | email           | telefono
-----
1 | Maria Alvarado | Quevedo | maria@uteq.edu.ec | 0991111111
2 | Luis Cedenó   | Babahoyo | luis@uteq.edu.ec  | 0992222222
3 | Ana Vera      | Quevedo | ana@uteq.edu.ec   | 0993333333
4 | Jose Mendoza  | Ventanas | jose@uteq.edu.ec   | 0994444444
5 | Carla Zambrano | Ventanas | carla@uteq.edu.ec  | 0995555555
6 | Pedro Suarez   | Babahoyo | pedro@uteq.edu.ec | 0996666666
(6 rows)

cafeteria=# select * from pedidos;
 pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----
1 | 1 | 1 | 2026-03-01 | 0.75 | Campus
2 | 2 | 3 | 2026-03-01 | 2.50 | Babahoyo
3 | 3 | 2 | 2026-03-02 | 1.00 | Campus
4 | 4 | 4 | 2026-03-02 | 1.25 | Ventanas
5 | 5 | 5 | 2026-03-03 | 1.00 | Ventanas
6 | 6 | 1 | 2026-03-03 | 0.75 | Babahoyo
7 | 1 | 3 | 2026-03-04 | 2.50 | Campus
8 | 2 | 2 | 2026-03-04 | 1.00 | Babahoyo
(8 rows)

cafeteria=# select * from pedidos;
 pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----
1 | 1 | 1 | 2026-03-01 | 0.75 | Campus
2 | 2 | 3 | 2026-03-01 | 2.50 | Babahoyo
3 | 3 | 2 | 2026-03-02 | 1.00 | Campus
4 | 4 | 4 | 2026-03-02 | 1.25 | Ventanas
5 | 5 | 5 | 2026-03-03 | 1.00 | Ventanas
6 | 6 | 1 | 2026-03-03 | 0.75 | Babahoyo
7 | 1 | 3 | 2026-03-04 | 2.50 | Campus
8 | 2 | 2 | 2026-03-04 | 1.00 | Babahoyo
(8 rows)

```

Figura 5: Verificación de los datos cargados en el nodo centralizado.

5. Fragmentación horizontal

La fragmentación horizontal divide la tabla **pedidos** por filas, usando la columna **sede** como criterio de corte. En álgebra relacional esto corresponde a una operación de selección (σ):

$$\text{pedidos_campus} = \sigma_{\text{sede}='Campus'}(\text{pedidos}) \quad (1)$$

$$\text{pedidos_babahoyo} = \sigma_{\text{sede}='Babahoyo'}(\text{pedidos}) \quad (2)$$

$$\text{pedidos_ventanas} = \sigma_{\text{sede}='Ventanas'}(\text{pedidos}) \quad (3)$$

Dado que los tres criterios son mutuamente excluyentes y colectivamente exhaustivos (cada pedido pertenece a exactamente una sede), se garantizan automáticamente las tres condiciones de corrección.

Listing 4: 03_fragmentacion_horizontal.sql — esquema aplicado en los tres nodos

```

1 CREATE TABLE pedidos_fragmento (
2     pedido_id INTEGER PRIMARY KEY,
3     cliente_id INTEGER NOT NULL,
4     producto_id INTEGER NOT NULL,
5     fecha DATE NOT NULL,
6     monto NUMERIC(8,2) NOT NULL,
7     sede VARCHAR(20) NOT NULL
8 );

```

Listing 5: 03_babahoyo.sql (ejemplo de inserción por nodo)

```

1 INSERT INTO pedidos_fragmento VALUES (2, 2, 3, '2026-03-01', 2.50, '
2     Babahoyo');
3 INSERT INTO pedidos_fragmento VALUES (6, 6, 1, '2026-03-03', 0.75, '
4     Babahoyo');
5 INSERT INTO pedidos_fragmento VALUES (8, 2, 2, '2026-03-04', 1.00, '
6     Babahoyo');

```

Se nombró la tabla de fragmento `pedidos_fragmento` (en lugar de `pedidos`) porque el nodo `pg-campus` ya contenía una tabla `pedidos` correspondiente al esquema centralizado de referencia. Usar el mismo nombre en los tres nodos evita el conflicto y mantiene consistencia en el diseño.

```

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-campus psql
pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----+-----+-----+-----+-----+-----
1 | 1 | 1 | 2026-03-01 | 0.75 | Campus
3 | 3 | 2 | 2026-03-02 | 1.00 | Campus
7 | 1 | 3 | 2026-03-04 | 2.50 | Campus
(3 rows)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container.
Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-babahoyo psql
pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----+-----+-----+-----+-----+-----
2 | 2 | 3 | 2026-03-01 | 2.50 | Babahoyo
6 | 6 | 1 | 2026-03-03 | 0.75 | Babahoyo
8 | 2 | 2 | 2026-03-04 | 1.00 | Babahoyo
(3 rows)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container.
Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-ventanas psql
pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----+-----+-----+-----+-----+-----
4 | 4 | 4 | 2026-03-02 | 1.25 | Ventanas
5 | 5 | 5 | 2026-03-03 | 1.00 | Ventanas
(2 rows)

```

Figura 6: Verificación de los fragmentos horizontales en los tres nodos (3, 3 y 2 filas respectivamente).

6. Fragmentación vertical

La fragmentación vertical divide la tabla `clientes` por columnas. El criterio no es geográfico sino de sensibilidad de los datos: se separan los atributos públicos (`nombre`, `ciudad`) de los atributos de contacto (`email`, `telefono`), que en un sistema real solo debería consultar el personal administrativo. En álgebra relacional esto es una proyección (π):

$$\text{clientes_publicos} = \pi_{\text{cliente_id}, \text{nombre}, \text{ciudad}}(\text{clientes}) \quad (4)$$

$$\text{clientes_contacto} = \pi_{\text{cliente_id}, \text{email}, \text{telefono}}(\text{clientes}) \quad (5)$$

Un detalle clave del diseño es que la clave primaria `cliente_id` se repite intencionalmente en ambos fragmentos: es la única excepción a la regla de disjunción, y es necesaria porque sin esa columna compartida sería imposible reconstruir la tabla original mediante un JOIN.

Listing 6: 04_campus.sql — fragmento público

```
1 CREATE TABLE clientes_publicos (  
2     cliente_id INTEGER PRIMARY KEY,  
3     nombre VARCHAR(80) NOT NULL,  
4     ciudad VARCHAR(40) NOT NULL  
5 );  
6  
7 INSERT INTO clientes_publicos VALUES  
8 (1, 'Maria Alvarado', 'Quevedo'),  
9 (2, 'Luis Cedenó', 'Babahoyo'),  
10 (3, 'Ana Vera', 'Quevedo'),  
11 (4, 'Jose Mendoza', 'Ventanas'),  
12 (5, 'Carla Zambrano', 'Ventanas'),  
13 (6, 'Pedro Suarez', 'Babahoyo');
```

Listing 7: 04_babahoyo.sql — fragmento de contacto

```
1 CREATE TABLE clientes_contacto (  
2     cliente_id INTEGER PRIMARY KEY,  
3     email VARCHAR(120) NOT NULL,  
4     telefono VARCHAR(20)  
5 );  
6  
7 INSERT INTO clientes_contacto VALUES  
8 (1, 'maria@uteq.edu.ec', '0991111111'),  
9 (2, 'luis@uteq.edu.ec', '0992222222'),  
10 (3, 'ana@uteq.edu.ec', '0993333333'),  
11 (4, 'jose@uteq.edu.ec', '0994444444'),  
12 (5, 'carla@uteq.edu.ec', '0995555555'),  
13 (6, 'pedro@uteq.edu.ec', '0996666666');
```

```

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-ca
cliente_id |      nombre      | ciudad
-----+-----+-----
1 | Maria Alvarado | Quevedo
2 | Luis Cedeno    | Babahoyo
3 | Ana Vera       | Quevedo
4 | Jose Mendoza   | Ventanas
5 | Carla Zambrano | Ventanas
6 | Pedro Suarez   | Babahoyo
(6 rows)

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-ba
cliente_id |      email      | telefono
-----+-----+-----
1 | maria@uteq.edu.ec | 0991111111
2 | luis@uteq.edu.ec  | 0992222222
3 | ana@uteq.edu.ec   | 0993333333
4 | jose@uteq.edu.ec  | 0994444444
5 | carla@uteq.edu.ec | 0995555555
6 | pedro@uteq.edu.ec | 0996666666
(6 rows)

```

Figura 7: Verificación de los fragmentos verticales: clientes_publicos (pg-campus) y clientes_contacto (pg-babahoyo).

7. Fragmentación mixta

La fragmentación mixta combina ambas técnicas sobre **clientes**: primero se corta horizontalmente por ciudad, y cada corte resultante se corta verticalmente por sensibilidad de los datos, según se resume en la Tabla 1.

Cuadro 1: Diseño de la fragmentación mixta de **clientes**.

Corte horizontal	Corte vertical	Columnas	Nodo
Ciudad = Quevedo	Datos públicos	cliente_id, nombre, ciudad	pg-campus
Ciudad = Quevedo	Datos contacto	cliente_id, email, telefono	pg-babahoyo
Ciudad = Babahoyo/Ventanas	Datos públicos	cliente_id, nombre, ciudad	pg-ventanas
Ciudad = Babahoyo/Ventanas	Datos contacto	cliente_id, email, telefono	pg-babahoyo

Un punto notable del diseño es que los dos cortes de datos de contacto (Quevedo y Babahoyo/-Ventanas) terminan físicamente en el mismo nodo (**pg-babahoyo**), aunque lógicamente sean fragmentos distintos. Por ello se crearon como dos tablas separadas (**clientes_contacto_quevedo** y **clientes_contacto_otras**) en ese nodo, preservando la trazabilidad del criterio de corte horizontal aunque compartan ubicación física.

Listing 8: 05_campus.sql — corte Quevedo, datos públicos

```

1 CREATE TABLE clientes_publicos_quevedo (
2     cliente_id INTEGER PRIMARY KEY,
3     nombre VARCHAR(80) NOT NULL,

```

```

4      ciudad VARCHAR(40) NOT NULL
5  );
6
7  INSERT INTO clientes_publicos_quevedo VALUES
8  (1, 'Maria Alvarado', 'Quevedo'),
9  (3, 'Ana Vera', 'Quevedo');

```

Listing 9: 05_babahoyo.sql — ambos cortes de contacto

```

1  CREATE TABLE clientes_contacto_quevedo (
2      cliente_id INTEGER PRIMARY KEY,
3      email VARCHAR(120) NOT NULL,
4      telefono VARCHAR(20)
5  );
6
7  INSERT INTO clientes_contacto_quevedo VALUES
8  (1, 'maria@uteq.edu.ec', '0991111111'),
9  (3, 'ana@uteq.edu.ec', '0993333333');
10
11 CREATE TABLE clientes_contacto_otras (
12     cliente_id INTEGER PRIMARY KEY,
13     email VARCHAR(120) NOT NULL,
14     telefono VARCHAR(20)
15 );
16
17 INSERT INTO clientes_contacto_otras VALUES
18 (2, 'luis@uteq.edu.ec', '0992222222'),
19 (4, 'jose@uteq.edu.ec', '0994444444'),
20 (5, 'carla@uteq.edu.ec', '0995555555'),
21 (6, 'pedro@uteq.edu.ec', '0996666666');

```

```

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-campus ps
cliente_id |      nombre      | ciudad
-----+-----+-----
          1 | Maria Alvarado   | Quevedo
          3 | Ana Vera         | Quevedo
(2 rows)

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-ventanas ps
cliente_id |      nombre      | ciudad
-----+-----+-----
          2 | Luis Cedeno      | Babahoyo
          4 | Jose Mendoza     | Ventanas
          5 | Carla Zambrano   | Ventanas
          6 | Pedro Suarez     | Babahoyo
(4 rows)

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-babahoyo ps
cliente_id |      email       | telefono
-----+-----+-----
          1 | maria@uteq.edu.ec | 0991111111
          3 | ana@uteq.edu.ec   | 0993333333
(2 rows)

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-babahoyo ps
cliente_id |      email       | telefono
-----+-----+-----
          2 | luis@uteq.edu.ec  | 0992222222
          4 | jose@uteq.edu.ec  | 0994444444
          5 | carla@uteq.edu.ec | 0995555555
          6 | pedro@uteq.edu.ec | 0996666666
(4 rows)

```

Figura 8: Verificación de los cuatro fragmentos de la fragmentación mixta.

8. Vistas globales con postgres_fdw

`postgres_fdw` (Foreign Data Wrapper) es una extensión nativa de PostgreSQL que permite que un nodo lea tablas de otro nodo remoto como si fueran locales, sin necesidad de mover físicamente los datos. En esta práctica, `pg-campus` actúa como nodo coordinador: declara a `pg-babahoyo` y `pg-ventanas` como servidores remotos, crea tablas espejo (`FOREIGN TABLE`) que apuntan a los fragmentos reales de cada nodo, y finalmente define dos vistas: `pedidos_global` (reconstrucción horizontal, mediante `UNION ALL`) y `clientes_global` (reconstrucción vertical, mediante `JOIN` por `cliente_id`).

Listing 10: 06_vistas_globales.sql

```

1 CREATE EXTENSION IF NOT EXISTS postgres_fdw;
2
3 CREATE SERVER srv_babahoyo
4   FOREIGN DATA WRAPPER postgres_fdw

```

```

5      OPTIONS (host 'pg-babahoyo', dbname 'cafeteria', port '5432');
6
7  CREATE SERVER srv_ventanas
8      FOREIGN DATA WRAPPER postgres_fdw
9      OPTIONS (host 'pg-ventanas', dbname 'cafeteria', port '5432');
10
11 CREATE USER MAPPING FOR admin SERVER srv_babahoyo
12     OPTIONS (user 'admin', password 'admin123');
13 CREATE USER MAPPING FOR admin SERVER srv_ventanas
14     OPTIONS (user 'admin', password 'admin123');
15
16 CREATE FOREIGN TABLE pedidos_babahoyo (
17     pedido_id INTEGER, cliente_id INTEGER, producto_id INTEGER,
18     fecha DATE, monto NUMERIC(8,2), sede VARCHAR(20)
19 ) SERVER srv_babahoyo OPTIONS (table_name 'pedidos_fragmento');
20
21 CREATE FOREIGN TABLE pedidos_ventanas (
22     pedido_id INTEGER, cliente_id INTEGER, producto_id INTEGER,
23     fecha DATE, monto NUMERIC(8,2), sede VARCHAR(20)
24 ) SERVER srv_ventanas OPTIONS (table_name 'pedidos_fragmento');
25
26 CREATE VIEW pedidos_global AS
27 SELECT * FROM pedidos_fragmento
28 UNION ALL
29 SELECT * FROM pedidos_babahoyo
30 UNION ALL
31 SELECT * FROM pedidos_ventanas;
32
33 CREATE FOREIGN TABLE clientes_contacto_remota (
34     cliente_id INTEGER, email VARCHAR(120), telefono VARCHAR(20)
35 ) SERVER srv_babahoyo OPTIONS (table_name 'clientes_contacto');
36
37 CREATE VIEW clientes_global AS
38 SELECT p.cliente_id, p.nombre, p.ciudad, c.email, c.telefono
39 FROM clientes_publicos p
40 JOIN clientes_contacto_remota c USING (cliente_id);

```

Se emplearon los nombres de contenedor (pg-babahoyo, pg-ventanas) como *host*, ya que Docker Compose los ubica automáticamente en la misma red interna, resolviéndose por nombre sin necesidad de direcciones IP.

```

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-campus psql
pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----+-----+-----+-----+-----+-----
1 | 1 | 1 | 2026-03-01 | 0.75 | Campus
2 | 2 | 3 | 2026-03-01 | 2.50 | Babahoyo
3 | 3 | 2 | 2026-03-02 | 1.00 | Campus
4 | 4 | 4 | 2026-03-02 | 1.25 | Ventanas
5 | 5 | 5 | 2026-03-03 | 1.00 | Ventanas
6 | 6 | 1 | 2026-03-03 | 0.75 | Babahoyo
7 | 1 | 3 | 2026-03-04 | 2.50 | Campus
8 | 2 | 2 | 2026-03-04 | 1.00 | Babahoyo
(8 rows)

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-campus psql
cliente_id | nombre      | ciudad   | email              | telefono
-----+-----+-----+-----+-----
1 | Maria Alvarado | Quevedo | maria@uteq.edu.ec | 0991111111
2 | Luis Ceden    | Babahoyo | luis@uteq.edu.ec  | 0992222222
3 | Ana Vera      | Quevedo | ana@uteq.edu.ec   | 0993333333
4 | Jose Mendoza  | Ventanas | jose@uteq.edu.ec  | 0994444444
5 | Carla Zambrano | Ventanas | carla@uteq.edu.ec | 0995555555
6 | Pedro Suarez   | Babahoyo | pedro@uteq.edu.ec | 0996666666
(6 rows)

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec -it pg-campus psql
sede;
sede | total
-----+-----
Babahoyo | 4.25
Campus | 4.25
Ventanas | 2.25

```

Figura 9: Consulta de pedidos_global, clientes_global y total de ventas por sede.

Esta última consulta demuestra el principio de **transparencia de fragmentación**: un usuario puede consultar `pedidos_global` y obtener totales agregados por sede sin saber que los datos están físicamente distribuidos en tres máquinas distintas.

9. Verificación de las tres condiciones

Listing 11: 07_verificacion.sql

```

1  -- 1) COMPLETITUD
2  SELECT COUNT(*) AS filas_globales FROM pedidos_global;
3  SELECT COUNT(*) AS filas_centralizadas FROM pedidos;
4
5  -- 2) RECONSTRUCCION
6  SELECT sede, SUM(monto) AS total_global
7  FROM pedidos_global GROUP BY sede ORDER BY sede;
8
9  SELECT sede, SUM(monto) AS total_centralizado
10 FROM pedidos GROUP BY sede ORDER BY sede;
11
12 -- 3) DISJUNCION

```

```

13 SELECT pedido_id, COUNT(*) AS veces
14 FROM pedidos_global
15 GROUP BY pedido_id
16 HAVING COUNT(*) > 1;

```

```

C:\Users\Usuario\Desktop\practica-fragmentacion>docker exec
  filas_globales
-----
              8
(1 row)

  filas_centralizadas
-----
              8
(1 row)

  sede | total_global
-----+-----
Babahoyo |      4.25
Campus   |      4.25
Ventanas |      2.25
(3 rows)

  sede | total_centralizado
-----+-----
Babahoyo |      4.25
Campus   |      4.25
Ventanas |      2.25
(3 rows)

pedido_id | veces
-----+-----
(0 rows)

```

Figura 10: Resultados de las consultas de verificación.

Los resultados confirman las tres condiciones de corrección:

- **Completitud:** `pedidos_global` y `pedidos` (centralizada) devuelven ambas 8 filas, confirmando que ningún dato se perdió al fragmentar.
- **Reconstrucción:** la suma de monto agrupada por sede es idéntica en ambas consultas (Babahoyo: 4.25, Campus: 4.25, Ventanas: 2.25), demostrando que `UNION ALL` reconstruye la tabla original sin alterar los valores.
- **Disjunción:** la consulta de `pedido_id` con `COUNT(*) > 1` devolvió cero filas, confirmando que ningún pedido está duplicado entre nodos.

10. Preguntas para pensar

1. **Si se abre una sede en El Empalme, ¿qué cambia y cuánto cuesta?** Solo se agrega un nuevo criterio $\sigma_{\text{sede}} = \text{'El Empalme'}$ y un nuevo nodo con su propio fragmento; no hay que tocar los fragmentos existentes. El costo es bajo porque el diseño horizontal ya está pensado para escalar por sede.
2. **¿Cuándo mejora la seguridad la fragmentación vertical?** Cuando datos sensibles como contraseñas o datos bancarios, entre otras, deben residir en un servidor con acceso más restringido que los datos generales; por ejemplo, separar el historial académico de un estudiante de sus datos de contacto público.
3. **¿Qué ventaja da replicar la PK en la fragmentación vertical?** Permite reconstruir la tabla original mediante un JOIN. Sin esa clave compartida sería imposible saber qué fila de un fragmento corresponde a qué fila del otro.
4. **¿Fue razonable fragmentar pedidos por sede en lugar de por producto_id?** Sí, porque las consultas típicas de una cafetería son del tipo “ventas de esta sede”, no “ventas de este producto”. Fragmentar por sede acerca los datos a quien los usa (localidad); por producto no aportaría esa ventaja en este escenario.
5. **¿Qué agregarías para tolerancia a fallos?** Agregaría la replicación, de manera que cada nodo tendría una copia secundaria en otro servidor, así si un nodo cae, sus datos sigan disponibles desde la réplica.

11. Reflexión

Esta práctica permitió comprobar que fragmentar una base de datos no es solo un ejercicio teórico sino que también exige tomar decisiones exactas sobre nombres de tablas, ubicación física de los datos y cómo reconstruirlos después. La principal dificultad fue un conflicto de nombres al reutilizar el nombre `pedidos` para el fragmento horizontal, este chocó con la tabla centralizada que ya existía en `pg-campus`, y en otro momento los scripts SQL quedaron vacíos por error, generando resultados que parecían correctos pero no reflejaban ninguna fragmentación real. Resolver ambos problemas, renombrando a `pedidos_fragmento` y verificando el contenido de cada archivo antes de ejecutarlo, refuerza la importancia de comprobar cada paso con consultas concretas en lugar de asumir que la ausencia de errores visibles significan un resultado correcto. La fragmentación mixta fue la parte más interesante porque mostró que un mismo nodo físico puede alojar varios fragmentos lógicos distintos, y que completitud, disjunción y reconstrucción deben verificarse en cada nivel de corte, no solo al final del proceso.

Referencias

- [1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Cham, Switzerland: Springer, 2020.
- [2] S. Ceri, S. B. Navathe, and G. Wiederhold, “Distribution design of logical database schemas,” *IEEE Transactions on Software Engineering*, vol. SE-9, no. 4, pp. 487–504, 1983.
- [3] S. B. Navathe, S. Ceri, G. Wiederhold, and J. Dou, “Vertical partitioning algorithms for database design,” *ACM Transactions on Database Systems*, vol. 9, no. 4, pp. 680–710, 1984.
- [4] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation — Table Partitioning.” Online documentation, 2024.